Python code from the project notebook:

```
# %%
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cvxpy as cp
from sklearn.preprocessing import StandardScaler

# %%
# Font for the plots
plt.rcParams['font.family'] = 'Avenir'

# %% [markdown]
# # Data prep

# %%
# Load Mystery Allocation 1 (MA1) and set date as index
df_myst1 = pd.read_csv('Mystery Allocation 1.csv', header = None, names = ['Date', 'MA1']).set_index('Date')
df_myst1.index.name = None

# Load Mystery Allocation 2 (MA2) and set date as index
df_myst2 = pd.read_csv('Mystery Allocation 2.csv', header = None, names = ['Date', 'MA2']).set_index('Date')
df_myst2.index.name = None

# Join Mystery Allocations into one df on date
df_myst = df_myst1.join(df_myst2, how = 'inner')

# Format date index
df_myst.index = pd.to_datetime(df_myst.index, format="%d/%m/%Y")

df_myst.head()

# %%
# Load Anonymized ETFs data, drop empty 1st row and set date as index
df_etf = pd.read_csv('Anonymized ETFs.csv').drop(index = 0).set_index('Unnamed: 0')

# Format date index
df_etf.index = pd.to_datetime(df_etf.index, format="%d/%m/%Y")
df_etf.index.name = None

df_etf.head()

# %%
# Load main data, drop empty 1st row and set date as index
df_main = pd.read_csv('Main Asset Classes.csv', header = 4).drop(index = 0).set_index('Unnamed: 0')

# Format date index
df_main.index = pd.to_datetime(df_main.index, format="%d/%m/%Y")
df_main.index.name = None

df_main.head()

# %%
# Show data range across datasets
print('Date range for anonymised ETFs:')
print(df_etf.index.min())
print(df_etf.index.max())

print('\n')
print('Date range for main assets:')
print(df_main.index.min())
print(df_main.index.max())

# %%
# Align time indices for the same period
df_etf_aligned, df_main_aligned = df_etf.align(df_main, 'inner', axis=0)
df_myst_aligned, df_main_aligned = df_myst.align(df_main, 'inner', axis=0)

# %%
# Check for missing data
print('NaNs in ETF Data Frame:', df_etf_aligned.isna().sum().max())
print('NaNs in Asset Data Frame:', df_main_aligned.isna().sum().max())
print('NaNs in Myster Allocation:', df_myst_aligned.isna().sum().max())

# %% [markdown]
# # Exploratory data analysis

# %% [markdown]
# ### Z-Scores for main asset prices

# %%
scaler = StandardScaler()

# Normalise price data for comparison
df_zscores = pd.DataFrame(scaler.fit_transform(df_main_aligned),
                          columns=df_main.columns,
                          index=df_main_aligned.index)

# Define broad asset classes for comparison
```

```
equities = ['S&P 500', 'Nasdaq 100', 'US Small Caps','Euro Stoxx 50', 'UK FTSE','MSCI EM', 'Japan ']
others = list(set(df_main_aligned.columns) - set(equities))

# Plot
plt.figure(figsize=(12,5), dpi=200)
plt.plot(df_zscores[equities], color = 'blue', linewidth=0.5)
plt.plot(df_zscores[others], color = 'red', linewidth=0.5)

plt.title("Z-scores of Main Asset Prices",fontsize=12)
plt.ylabel("Z-score")

from matplotlib.lines import Line2D
custom_legend = [
    Line2D([0], [0], color='blue', lw=1, label='Equities'),
    Line2D([0], [0], color='red', lw=1, label='Other Assets')]

plt.legend(handles=custom_legend, loc='upper left', fontsize=8)
plt.tight_layout()
plt.show()

# %% [markdown]
# ### Z-scores for anonymised ETFs

# %%
# Normalise ETF prices for comparison
df_zscores_etf = pd.DataFrame(scaler.fit_transform(df_etf_aligned),
                          columns=df_etf_aligned.columns,
                          index=df_etf_aligned.index)

# Plot
plt.figure(figsize=(12,5), dpi=200)
plt.plot(df_zscores_etf, linewidth=0.5, color='black')
plt.title("Z-scores of Anonymised ETFs",fontsize=12)
plt.ylabel("Z-score")
plt.tight_layout()
plt.show()

# %%
# Identified outlier ETFs
outliers = ['ETF 72','ETF 64', 'ETF 48', 'ETF 46']

# Drop outleirs in final ETF data frame
df_etf_aligned = df_etf.drop(columns=outliers)

# Outliers plot
df_etf[outliers + ['ETF 1 ']].plot() # ETF 1 for comparison

# %% [markdown]
# ### Key stats for main assets

# %%
df_main_lreturns = np.log(df_main_aligned / df_main_aligned.shift(1)).dropna()
df_stats = pd.DataFrame(index=df_main_lreturns.columns)

df_stats['Mean Daily Return'] = df_main_lreturns.mean()
df_stats['Annualized Return'] = df_main_lreturns.mean() * 252
df_stats['Daily Volatility'] = df_main_lreturns.std()
df_stats['Annualized Volatility'] = df_main_lreturns.std() * np.sqrt(252)
df_stats['Sharpe Ratio'] = df_stats['Annualized Return'] / df_stats['Annualized Volatility']
df_stats['Max Drawdown'] = (df_main_lreturns.cumsum() - df_main_lreturns.cumsum().cummax()).min()
df_stats['Skewness'] = df_main_lreturns.skew()
df_stats['Kurtosis'] = df_main_lreturns.kurtosis()

df_stats = df_stats.round(4)
df_stats

# %%
# Choose key metrics to plot
metrics = ['Annualized Return', 'Annualized Volatility', 'Sharpe Ratio', 'Max Drawdown']
df_plot = df_stats[metrics]

# Setup positions for horizontal bars
n_assets = len(df_plot)
bar_height = 0.2
y_pos = np.arange(n_assets)

plt.figure(figsize=(11,5), dpi=200)

# Plot each metric as a horizontal bar with offset
for i, metric in enumerate(metrics):
    plt.barh(y_pos + i*bar_height, df_plot[metric], height=bar_height, label=metric)

plt.yticks(y_pos + bar_height*(len(metrics)/2 - 0.5), df_plot.index)
plt.xlabel('Value')
plt.title('Asset Class Key Metrics', fontsize = 12)
plt.legend(fontsize=8, loc = 'upper left')
plt.tight_layout()
plt.show()

# %% [markdown]
# # Variable calculation
```

```
181
182 # %%
183 # Daily log returns
184 df_lreturns = np.log(df_etf_aligned / df_etf_aligned.shift(1)).dropna()
185 df_main_lreturns = np.log(df_main_aligned / df_main_aligned.shift(1)).dropna()
186 df_myst_lreturns = np.log(df_myst_aligned / df_myst_aligned.shift(1)).dropna()
187
188 # %%
189 # Daily cummulative log returns
190 df_lreturns_cum = df_lreturns.cumsum()
191 df_main_lreturns_cum = df_main_lreturns.cumsum()
192
193 # %%
194 # Daily mean returns
195 df_lreturns_mean = df_lreturns.mean()
196 df_main_lreturns_mean = df_main_lreturns.mean()
197
198 # %%
199 # Annualised returns
200 df_lreturns_annual = df_lreturns_mean*252
201 df_main_lreturns_annual = df_main_lreturns_mean*252
202
203 # %%
204 # Standard deviation of daily returns
205 df_vol = df_lreturns.std()
206 df_main_vol = df_main_lreturns.std()
207
208 # %%
209 # Standard deviation of annualised returns
210 df_vol_annual = df_vol * np.sqrt(252)
211 df_main_vol_annual = df_main_vol * np.sqrt(252)
212
213 # %%
214 # Sharpe ratio
215 df_sharpe_ratio = df_lreturns_annual / df_vol_annual
216 df_main_sharpe_ratio = df_main_lreturns_annual / df_main_vol_annual
217
218 # %%
219 # Sortino ratio
220
221 # Downside (negative) returns only
222 downside_returns = df_lreturns_annual[df_lreturns_annual < 0]
223
224 # Downside standard deviation
225 downside_std_annual = np.std(downside_returns) * np.sqrt(252)
226
227 # Caclulate Ratio
228 df_sortino_ratio = df_lreturns_annual / downside_std_annual
229
230 # %%
231 # Max Drawdown
232 max_drawdown = (df_lreturns_cum - df_lreturns_cum.cummax()).min().abs()
233
234 # %%
235 # Descriptive stats for key metrics
236
237 df_stats_etf = pd.DataFrame({
238     'Returns (an.)' : df_lreturns_annual,
239     'Volatility (an.)' : df_vol_annual,
240     'Sharpe' : df_sharpe_ratio,
241     'Sortino' : df_sortino_ratio,
242     'Max Drawdown' : max_drawdown
243 })
244
245 df_stats_etf.describe().round(4)
246
247 # %% [markdown]
248 # # Performance & risk metrics
249
250 # %% [markdown]
251 # ### Cummulative returns (log)
252
253 # %%
254 plt.figure(figsize=(11,5), dpi=200)
255
256 plt.plot(df_lreturns_cum, color='grey', linewidth=0.8)  # single color line
257 plt.plot(df_lreturns_cum.median(axis=1), color = 'red')
258
259 custom_legend = [Line2D([0], [0], color='red', lw=2, label='median')]
260 plt.legend(handles=custom_legend, loc='upper left', fontsize=10)
261
262 plt.title("Cummulative Log Return on ETFs", fontsize=12)
263 plt.ylabel("Log Return")
264
265 plt.grid(True, linestyle='--', alpha=0.5, axis='y')
266
267 plt.tight_layout()
268 plt.show()
269
270 # %%
271 plt.figure(figsize=(11,5), dpi=200)
```

```
272
273 sns.histplot(
274     df_lreturns_cum.iloc[-1],
275     bins=20,
276     stat='count',
277     color='blue',
278     alpha=0.4,
279     kde=True
280 )
281
282 # Calculate statistics to display
283 mean = df_lreturns_cum.iloc[-1].mean()
284 median = df_lreturns_cum.iloc[-1].median()
285 std = df_lreturns_cum.iloc[-1].std()
286
287 plt.title('Distribution of ETF Total Log Returns', )
288 plt.xlabel('Total Return')
289
290 x_min, x_max = plt.xlim()
291 plt.xticks(np.arange(np.floor(x_min*4)/4, np.ceil(x_max*4)/4 + 0.25, 0.25))
292
293 # Add vertical lines
294 plt.axvline(median, color='red', linestyle='--', linewidth=2, label='Median')
295 plt.axvline(mean, color='black', linestyle='--', linewidth=2, label='Mean')
296 plt.axvline(mean + std, color='orange', linestyle='--', linewidth=2, label='+/- 1 SD')
297 plt.axvline(mean - std, color='orange', linestyle='--', linewidth=2)
298 plt.legend(fontsize=10)
299
300
301 plt.tight_layout()
302 plt.show()
303
304 # %% [markdown]
305 # ### Risk-adjusted returns (log)
306
307 # %%
308 risk_adjusted_lreturns = (df_lreturns_cum / df_vol_annual)
309
310 plt.figure(figsize=(11,5), dpi=200)
311 plt.plot(risk_adjusted_lreturns, color='grey', linewidth=0.8)
312 plt.plot(risk_adjusted_lreturns.median(axis=1), color = 'red')
313
314 plt.title("Risk-Adjusted Cumulative Log Return", fontsize=12)
315 plt.ylabel("Risk-Adjusted Return")
316 plt.grid(True, linestyle='--', alpha=0.5, axis='y')
317
318 custom_legend = [Line2D([0], [0], color='red', lw=2, label='median')]
319 plt.legend(handles=custom_legend, loc='upper left', fontsize=10)
320
321 plt.tight_layout()
322 plt.show()
323
324 # %%
325 plt.figure(figsize=(11,5), dpi=200)
326
327 sns.histplot(
328     risk_adjusted_lreturns.iloc[-1],
329     bins=20,
330     stat='count',
331     color='blue',
332     alpha=0.4,
333     kde=True
334 )
335
336 # Calculate statistics to display
337 mean = risk_adjusted_lreturns.iloc[-1].mean()
338 median = risk_adjusted_lreturns.iloc[-1].median()
339 std = risk_adjusted_lreturns.iloc[-1].std()
340
341 plt.title('Distribution of ETF Risk-Adjusted Returns', )
342 plt.xlabel('Total Return')
343
344 x_min, x_max = plt.xlim()
345 plt.xticks(np.arange(np.floor(x_min*4)/4, np.ceil(x_max*4)/4 + 0.25, 0.25))
346
347 # Add vertical lines
348 plt.axvline(median, color='red', linestyle='--', linewidth=2, label='Median')
349 plt.axvline(mean, color='black', linestyle='--', linewidth=2, label='Mean')
350 plt.axvline(mean + std, color='orange', linestyle='--', linewidth=2, label='+/- 1 SD')
351 plt.axvline(mean - std, color='orange', linestyle='--', linewidth=2)
352 plt.legend(fontsize=10)
353
354
355 plt.tight_layout()
356 plt.show()
357
358 # %% [markdown]
359 # ### Annualised metrics
360
361 # %%
362 fig, axes = plt.subplots(1, 2, figsize=(11,5), dpi=200)
```

```
363
364 sns.histplot(
365     df_lreturns_annual,
366     bins=20,
367     stat='count',
368     color='blue',
369     alpha=0.4,
370     kde=True,
371     ax=axes[0]
372 )
373 axes[0].set_title('Annualised Log Return', fontsize = 10)
374 axes[0].set_ylabel('Count')
375
376 sns.histplot(
377     df_vol_annual,
378     bins=20,
379     stat='count',
380     color='green',
381     alpha=0.4,
382     kde=True,
383     ax=axes[1]
384 )
385 axes[1].set_title('Annualised Volatility', fontsize = 10)
386 axes[1].set_ylabel('')
387
388 plt.tight_layout()
389 plt.show()
390
391
392 # %%
393 plt.figure(figsize=(10,5),dpi=200)
394 plt.scatter(x = df_lreturns_annual, y = df_vol_annual, alpha=0.5, color='blue')
395
396 plt.xlabel('Annualised Return', fontsize = 12)
397 plt.ylabel('Annualised Volatility',fontsize = 12)
398 plt.title('Risk-Return Relationship for ETFs', fontsize = 12)
399
400 plt.tight_layout()
401 plt.show()
402
403 # %%
404 # Correlation in clusters
405
406 high_vol = df_vol_annual.loc[df_vol_annual>0.1].index
407
408 df = pd.DataFrame({
409     'vol' : df_vol_annual.loc[high_vol],
410     'ret': df_lreturns_annual.loc[high_vol]
411 })
412
413 df.loc[df['ret'] < 0.17].corr()
414
415 # %%
416 fig, axes = plt.subplots(1, 2, figsize=(10,5), dpi=200)
417
418 sns.scatterplot(
419     x = df_lreturns_annual,
420     y = max_drawdown,
421     ax=axes[0],
422     c='blue',
423     alpha = 0.5
424 )
425
426 axes[0].set_ylabel('Max Drawdown')
427 axes[0].set_xlabel('Annualised Log Return')
428
429 sns.scatterplot(
430     x = df_vol_annual,
431     y = max_drawdown,
432     ax=axes[1],
433     c='blue',
434     alpha = 0.5
435 )
436
437 axes[1].set_ylabel('')
438 axes[1].set_xlabel('Annualised Volatility')
439
440 fig.suptitle('ETF  R i s k Reward  Profile: Annualized Metrics vs Max Drawdown', fontsize=12)
441
442 plt.tight_layout()
443 plt.show()
444
445
446 # %% [markdown]
447 # ### Sharpe + Sortino
448
449 # %%
450 df_box = pd.DataFrame({
451     'Sharpe Ratio': df_sharpe_ratio,
452     'Sortino Ratio': df_sortino_ratio
453 })
```

```
454
455 # Melt the DataFrame to long format for Seaborn
456 df_melt = df_box.melt(var_name='Metric', value_name='Value')
457
458 plt.figure(figsize=(8,5), dpi=200)
459
460 sns.boxplot(x='Metric', y='Value', data=df_melt, palette='Set2')
461
462 plt.title('Distribution of ETF Performance Metrics')
463 plt.ylabel('Value')
464 plt.xlabel('')
465 plt.grid(axis='y', linestyle='--', alpha=0.7)
466 plt.tight_layout()
467 plt.show()
468
469 # %%
470 fig, axes = plt.subplots(1, 2, figsize=(11,5), dpi=200)
471
472 # --- Sharpe ratio ---
473 sns.histplot(
474     df_sharpe_ratio.loc[df_sharpe_ratio.between(-2, 2)],
475     bins=20,
476     stat='count',
477     color='blue',
478     alpha=0.4,
479     kde=True,
480     ax=axes[0]
481 )
482 axes[0].set_title('Sharpe Ratio', fontsize = 10)
483 axes[0].set_ylabel('Count')
484
485 # --- Sortino ratio ---
486 sns.histplot(
487     df_sortino_ratio,
488     bins=20,
489     stat='count',
490     color='green',
491     alpha=0.4,
492     kde=True,
493     ax=axes[1]
494 )
495 axes[1].set_title('Sortino Ratio', fontsize = 10)
496 axes[1].set_ylabel('')
497
498 plt.tight_layout()
499 plt.show()
500
501
502 # %%
503 sharpe_trimmed = df_sharpe_ratio.loc[df_sharpe_ratio.between(-2,2)]
504 sortino_trimmed = df_sortino_ratio[sharpe_trimmed.index]
505
506 plt.figure(figsize=(11,5),dpi=200)
507 plt.scatter(x = sharpe_trimmed, y = sortino_trimmed, alpha=0.5, color='blue')
508
509 min_val = min(sharpe_trimmed.min(), sortino_trimmed.min())
510 max_val = max(sharpe_trimmed.max(), sortino_trimmed.max())
511
512 # Plot 45-degree line
513 plt.plot(
514     [min_val-0.2, max_val],
515     [min_val, max_val],
516     linestyle='--',
517     color='black',
518     linewidth=1,
519     label='45   line (Sharpe = Sortino)'
520 )
521
522 custom_legend = [Line2D([0], [0], color='black', linestyle='--',lw=1, label='Sharpe = Sortino')]
523 plt.legend(handles=custom_legend, loc='upper left', fontsize=10)
524
525 plt.xlim(-0.5, 1.2)
526 plt.xlabel('Sharpe Ratio', fontsize = 10)
527 plt.ylabel('Sortino Ratio',fontsize = 10)
528 plt.title('Risk-Adjusted Performance: Sharpe vs Sortino Ratios', fontsize = 12)
529
530 plt.tight_layout()
531 plt.show()
532
533 # %% [markdown]
534 # ### Summary (top / bottom 5)
535
536 # %%
537 df_lreturns_cum.iloc[-1].sort_values(ascending=False)
538
539 # %%
540 df_vol_annual.sort_values(ascending=False)
541
542 # %%
543 df_sharpe_ratio.sort_values(ascending=False)
544
```

```
# %%
df_sortino_ratio.sort_values(ascending=False)

# %% [markdown]
# # Relation analysis

# %%
df_combined = pd.concat([df_etf_aligned, df_main_aligned], axis=1)
corr_matrix = df_combined.corr().loc[df_etf_aligned.columns, df_main_aligned.columns]

# %% [markdown]
# ### Correlation

# %%
plt.figure(figsize=(20, 15))
sns.heatmap(corr_matrix.sort_values(by='S&P 500', ascending=False),
            cmap="coolwarm",     # red/blue diverging colors
            center=0,            # 0 = white midpoint
            annot=False)         # set to True if you want numbers
plt.title("Correlation Heatmap (sorted by: S&P 500)", fontsize=14)
plt.show()


# %% [markdown]
# ### Linear Regression

# %%
# Data Frame for both ETFs and MAs
df_lreturns_all = pd.concat([df_lreturns, df_myst_lreturns], axis=1)

import statsmodels.api as sm

# Empty frames to store regression parameters
betas = pd.DataFrame(index=df_lreturns_all.columns,
                     columns=df_main_aligned.columns)

pvals = pd.DataFrame(index=df_lreturns_all.columns,
                     columns=df_main_aligned.columns)

contrib = pd.DataFrame(index=df_lreturns_all.columns,
                       columns=df_main_aligned.columns)

# Empty dictionary to store models
models = {}

for etf in df_lreturns_all.columns:
    y = df_lreturns_all[etf]
    X = sm.add_constant(df_main_lreturns)
    model = sm.OLS(y, X).fit()


    # store betas, p-values (exclude constant term) and models
    betas.loc[etf] = model.params.drop('const')
    pvals.loc[etf] = model.pvalues.drop('const')
    models[etf] = model
    contrib.loc[etf] = model.params.drop('const') * df_lreturns_all[etf].sum()

# %%
# Filter out isignificant betas
significance = (pvals <= 0.05).astype(float) # 1 = significant, 0 otherwise
betas_signif = (betas * significance).astype(float)

# %%
plt.figure(figsize=(20, 15))
sns.heatmap(significance,
            cmap="coolwarm",     # red/blue diverging colors
            center=0,            # 0 = white midpoint
            annot=False)         # set to True if you want numbers
plt.title("Regression coefficients")
plt.show()

# %%
plt.figure(figsize=(20, 15))
sns.heatmap(betas_signif.sort_values(by=['S&P 500'], ascending=False),
            cmap="coolwarm",     # red/blue diverging colors
            center=0,            # 0 = white midpoint
            annot=False,         # set to True if you want numbers
            vmin=-0.2,
            vmax=0.2)
plt.title("Regression coefficients (significant only; trimmed colour scale; sorted by: S&P 500)", fontsize
      =14)
plt.show()

# %%
# OLS-based contributions for a stacked barchart
total_returns = (contrib).sum(axis=1)
contrib_sorted = (contrib).loc[total_returns.sort_values(ascending=False).index]

import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(11,5), dpi=200)
```

```

# Stacked bar chart
contrib_sorted.plot(kind='bar', stacked=True, ax=ax, cmap='tab20', width=0.7)

ax.plot(
    np.arange(len(contrib_sorted)),  # x positions
    contrib.sum(axis=1).sort_values(ascending=False),           # y values
    color='black',
    linewidth=0.5,
    marker='o',
    label='Fitted Total Return'
)

# Labels and title
ax.set_ylabel("Total Log Return")
ax.set_title("Regression-Based Contributions of Main Assets to ETF Returns", fontsize=12)
plt.xticks(rotation=90, fontsize=6)

# Grid and legend
ax.grid(axis='y', linestyle='--', alpha=0.5)
ax.legend(loc = 'upper right', ncol = 2, fontsize=8)

plt.tight_layout()
plt.show()

# %% [markdown]
# ### PCA

# %% [markdown]
# Principal Component Loadings

# %%
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X = df_main_lreturns.copy()
etf_returns = df_lreturns_all.copy()

# Standardize main asset returns
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Run PCA on asset classes
pca = PCA()
factor_returns_scaled = pca.fit_transform(X_scaled)

# Create loadings DataFrame (components)
loadings = pd.DataFrame(
    pca.components_,
    columns=X.columns,
    index=[f"PC{i+1}" for i in range(len(X.columns))]
)

# %% [markdown]
# Variance explained by PCs

# %%
explained_variance = pd.Series(pca.explained_variance_ratio_, index=loadings.index)

plt.figure(figsize=(11,5), dpi=200)
plt.plot(range(1, len(explained_variance)+1), explained_variance, marker='o', c='blue')

plt.xlabel("Principal Component")
plt.ylabel("Explained Variance Ratio")
plt.title("Scree Plot", fontsize = 12)
plt.grid(True)

plt.tight_layout()
plt.show()

# %% [markdown]
# Loadings magnitude chart

# %%
fig, ax = plt.subplots(figsize=(11,5), dpi=200)

# Plot first 5 PCs as bar chart
loadings.iloc[:5, :].plot(kind='bar', ax=ax, width=0.6, cmap='tab20b')

ax.set_ylabel('Loading')
ax.set_title('PCA Loadings: First 5 Principal Components', fontsize = 12)

plt.xticks(rotation=0)
ax.grid(axis='y', linestyle='--', alpha=0.5)

ax.legend(bbox_to_anchor=(1.05, 0.9), loc='upper left')

plt.tight_layout()
plt.show()

# %% [markdown]
```

```
# PCA regression betas

# %%
etf_returns = df_lreturns_all.copy()

# Convert factor returns back to real scale
factor_returns_real = X.values @ pca.components_.T
factor_returns_real = pd.DataFrame(
    factor_returns_real,
    index=X.index,
    columns=loadings.index
)

factor_cum = factor_returns_real.cumsum()

top_pcs = ['PC1','PC2','PC3','PC4','PC5']  # choose top 5 PCs

betas_pca = {}

for etf in etf_returns.columns:
    y = etf_returns[etf]
    X_f = sm.add_constant(factor_returns_real[top_pcs])
    model = sm.OLS(y, X_f).fit()
    betas_pca[etf] = model.params

betas_pca = pd.DataFrame(betas_pca).T

# %% [markdown]
# Return contributions

# %%
# Total cumulative factor returns
total_factor_return = factor_cum[top_pcs].iloc[-1]  # final value per PC

# Contribution = beta * factor cumulative return
contrib_pca = betas_pca.drop(columns='const').multiply(total_factor_return, axis=1)

# Compute total contribution per ETF and sort
sorted_order = df_lreturns_cum.iloc[-1].sort_values(ascending=False).index
contrib_pca_sorted = contrib_pca.loc[sorted_order]

# Plot
fig, ax = plt.subplots(figsize=(11,5), dpi=200)

contrib_pca_sorted.plot(
    kind='bar',
    stacked=True,
    ax=ax,
    cmap='tab20',
    width=0.8
)

plt.xticks(rotation=90, fontsize=6)

ax2 = ax.twinx()
ax2.plot(
    np.arange(len(sorted_order)),
    df_lreturns_cum.iloc[-1].sort_values(ascending=False).values,
    color='black',
    linewidth=2,
    marker='o',
    label='Total Return'
)

ax.set_title("ETF Return Attribution by PCA Factors", fontsize=12)
ax.set_ylabel("Contribution to Total Return")
ax2.set_ylabel("Total Log Return")

ax.grid(axis='y', linestyle='--', alpha=0.5)
ax.legend(loc='upper right', fontsize=9)


plt.tight_layout()
plt.show()

# %% [markdown]
# Variance contributions

# %%
# PC variances
pc_variance = factor_returns_real[top_pcs].var()

# Normalised variance contrib
var_contrib_raw = betas_pca[top_pcs]**2 * pc_variance
var_contrib_pct = var_contrib_raw.div(var_contrib_raw.sum(axis=1),axis=0)
sorted_order = df_lreturns_cum.iloc[-1].sort_values(ascending=False).index

# Plot
fig, ax = plt.subplots(figsize=(11,6), dpi=200)

var_contrib_pct.loc[sorted_order].plot(
```

```
    kind='bar',
    stacked=True,
    ax=ax,
    cmap='tab20',
    width=0.8
)

ax.set_title("ETF Volatility Attribution by PCA Factors")
ax.set_ylabel("Contribution to Total Variance")
plt.xticks(rotation=90, fontsize=6)
ax.grid(axis='y', linestyle='--', alpha=0.5)
ax.legend(title='PC', bbox_to_anchor=(1.05,1), loc='upper left')

ax2 = ax.twinx()
ax2.plot(
    np.arange(len(sorted_order)),
    df_lreturns_cum.iloc[-1].sort_values(ascending=False).values,
    color='black',
    linewidth=2,
    marker='o',
    label='Total Return'
)

ax2.set_ylabel("Total Log Return")


plt.tight_layout()
plt.show()


# %% [markdown]
# Asset class identifications

# %%
total_returns = df_lreturns_cum.iloc[-1]

top_etfs = total_returns.nlargest(5).index
bot_etfs = total_returns.nsmallest(5).index
middle_etfs = (
    total_returns
    .sort_values()
    .iloc[len(total_returns)//2 - 2 : len(total_returns)//2 + 3]
    .index)
mystery_etfs = ['MA1', 'MA2']

etf_groups = {
    'Top Performing' : top_etfs,
    'Mid Performing' : middle_etfs,
    'Worst Performing' : bot_etfs,
    'Mystery Allocation' : mystery_etfs
}

# %%
chosen_group = 'Worst Performing' # chose amoung defined above

fig, axes = plt.subplots(2, 1, figsize=(12,9), dpi=200, sharey=True)

sns.heatmap(
    betas_pca[top_pcs].loc[etf_groups[chosen_group]],
    center=0,
    cmap="coolwarm",
    annot=True,
    fmt=".2f",
    linewidths=0.5,
    linecolor='white',
    cbar_kws={'label': 'Factor Loading'},
    ax=axes[0]
)

axes[0].set_title(f"Factor Loadings of {chosen_group} ETFs on Principal Components", fontsize=12)


sns.heatmap(betas_signif.loc[etf_groups[chosen_group]],
            cmap="coolwarm",     # red/blue diverging colors
            center=0,            # 0 = white midpoint
            annot=True,
            fmt=".2f",
            linewidths=0.5,
            linecolor='white',
            cbar_kws={'label': 'Regression Coef.'},
            vmin=-0.5,
            vmax=0.5,
            ax=axes[1])

axes[1].set_title(f"Regression Coefficients (OLS) of {chosen_group} Performing ETFs", fontsize=12)

plt.tight_layout() # Adjust layout to prevent labels from being cut off
plt.show()

# %% [markdown]
# # Mystery allocations
```

```
# %% [markdown]
# ### Static

# %%
# Inputs
y = df_lreturns_all['MA1'].values
X = df_lreturns.values

n_assets = X.shape[1]

# Optimization variable
w = cp.Variable(n_assets)

# Objective: minimize tracking error
objective = cp.Minimize(cp.sum_squares(y - X @ w))

# Constraints
constraints = [
    w >= 0,
    cp.sum(w) == 1
]

problem = cp.Problem(objective, constraints)
problem.solve()

# Results
weights = pd.Series(w.value, index=df_lreturns.columns)
weights.round(2).abs().sort_values(ascending=False).loc[weights >0.01]

# %% [markdown]
# ### Dynamic

# %%
window = 252  # 1 year
assets = df_lreturns
etf = df_lreturns_all['MA2']

weights_rolling = []

dates = assets.index[window:]

for end in range(window, len(assets)):
    X = assets.iloc[end-window:end].values
    y = etf.iloc[end-window:end].values

    w = cp.Variable(X.shape[1])

    prob = cp.Problem(
        cp.Minimize(cp.sum_squares(y - X @ w)),
        [w >= 0, cp.sum(w) == 1]
    )
    prob.solve(solver=cp.OSQP)

    weights_rolling.append(w.value)

weights_rolling = pd.DataFrame(
    weights_rolling,
    index=dates,
    columns=assets.columns
)

# %%
import matplotlib.pyplot as plt

colors = plt.get_cmap('tab20c').colors  # returns 20 RGB tuples

plt.figure(figsize=(11,5), dpi=200)

for i, col in enumerate(weights_rolling.columns):
    plt.plot(weights_rolling.index, weights_rolling[col], label=col, color=colors[i % 20])

plt.title('Rolling Implied Asset Allocation For Dynamic Portfolio', fontsize=12)
plt.ylabel("Weight")
# plt.legend(loc='upper right', ncol=2, fontsize = 8)
plt.tight_layout()
plt.show()


# %% [markdown]
# Noise filtering

# %%
# 1. Define the approximate "flat" periods based on visual inspection
periods_of_interest = [
    ('2020-03-01', '2020-10-01'),  # Period 1: Greens/Brown/Lavender
    ('2021-09-01', '2022-09-01'),  # Period 2: Lavender/Green/Orange
    ('2023-06-01', '2024-05-01')   # Period 3: Violet/Blue
]

# 2. Identify the Key ETFs
```

```python
significant_etfs = set()

for start_date, end_date in periods_of_interest:
    # Slice the data for the specific period
    period_data = weights_rolling.loc[start_date:end_date]

    # Calculate average allocation in this window
    avg_weights = period_data.mean()

    # Select ETFs with > 5% allocation in this period
    leaders = avg_weights[avg_weights > 0.05].index.tolist()

    # Add to our master list
    significant_etfs.update(leaders)

    print(f"Leaders for {start_date} to {end_date}: {leaders}")

# 3. Filter the Main DataFrame
clean_weights_rolling = weights_rolling[list(significant_etfs)]

# 4. Plot with DPI = 200
# We create the figure and axes explicitly to control DPI
fig, ax = plt.subplots(figsize=(11, 5), dpi=200)

clean_weights_rolling.plot(
    ax=ax,
    title="Filtered Dynamic Allocation (Dominant Regimes Only)"
)

plt.ylabel("Weight")
plt.xlabel("Date")
plt.legend(loc='center right', fontsize=8)
plt.tight_layout() # Adjust layout to prevent clipping of the legend
plt.show()

# 5. Display the final list
print("\nFinal Filtered ETF List:", list(significant_etfs))

# %% [markdown]
# Fitted allocation

# %%
# 1. Define the Stable Regimes
# (Adjusted slightly to ensure coverage, but your dates are fine)
periods_of_interest = [
    ('2020-03-01', '2020-10-01'),
    ('2021-09-01', '2022-09-01'),
    ('2023-06-01', '2024-05-01')
]

# 2. Identify the Universe of Dominant ETFs
significant_etfs = set()
for start, end in periods_of_interest:
    period_mean = weights_rolling.loc[start:end].mean()
    significant_etfs.update(period_mean[period_mean > 0.05].index.tolist())

# 3. Create the "Fitted" Series
fitted_allocation = pd.DataFrame(0.0, index=weights_rolling.index, columns=list(significant_etfs))

print("--- FITTED ALLOCATION MODEL ---")

# 4. Fill with Average Weights (THE FIX)
for i, (start, end) in enumerate(periods_of_interest):
    # Get actual data for this window
    period_data = weights_rolling.loc[start:end, list(significant_etfs)]

    # Calculate the average weight
    avg_weights = period_data.mean()
    active_weights = avg_weights[avg_weights > 0.05]

    # --- FIX START: Assign column by column to avoid broadcasting errors ---
    for etf, weight in active_weights.items():
        fitted_allocation.loc[start:end, etf] = weight
    # --- FIX END ---

    print(f"\nPeriod {i+1} ({start} to {end}):")
    print(active_weights.round(3).to_string())

# 5. Fill Gaps (Optional but Recommended)
# The fitted_allocation currently has 0.0s between your periods (e.g. late 2020 to late 2021).
# Forward filling assumes the strategy stays static until the next regime change.
fitted_allocation = fitted_allocation.replace(0.0, pd.NA).ffill().fillna(0.0)

# ------------------------------------------------------------
# CALCULATION PART
# ------------------------------------------------------------

# Align dates
aligned_assets = df_lreturns.loc[fitted_allocation.index]
aligned_ma2 = df_lreturns_all.loc[fitted_allocation.index, 'MA2']

```

```
# Calculate Reconstructed Returns
# Using .sum(min_count=1) prevents 0s if data is missing, but normal sum is fine here
reconstructed_returns = (fitted_allocation * aligned_assets).sum(axis=1)

# Calculate Cumulative Performance
cum_reconstructed = reconstructed_returns.cumsum()
cum_ma2 = aligned_ma2.cumsum()

# Plot Comparison
fig, ax = plt.subplots(figsize=(11, 5), dpi=200)
ax.plot(cum_ma2.index, cum_ma2, label='Original MA2 (Mystery)', color='black', linewidth=2, alpha=0.7)
ax.plot(cum_reconstructed.index, cum_reconstructed, label='Reconstructed Strategy (Fitted)', color='red',
        linestyle='--', linewidth=1.2)

ax.set_title('Fitted Dynamic Allocation vs. Original MA2')
ax.set_ylabel('Cumulative Log Returns')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Correlation
print(f"Correlation: {reconstructed_returns.corr(aligned_ma2):.4f}")

# %% [markdown]
# Goodness-off-fit measures

# %%
from sklearn.metrics import r2_score, mean_squared_error

# ------------------------------------------------------------
# 1. Define Metrics Function
# ------------------------------------------------------------
def get_fit_metrics(true_returns, model_returns, model_name):
    # Align data to be safe
    common_idx = true_returns.index.intersection(model_returns.index)
    true_s = true_returns.loc[common_idx]
    model_s = model_returns.loc[common_idx]

    # Calculate Residuals (Tracking Difference)
    residuals = true_s - model_s

    # 1. Correlation
    corr = true_s.corr(model_s)

    # 2. R-Squared (Goodness of Fit)
    r2 = r2_score(true_s, model_s)

    # 3. RMSE (Root Mean Squared Error) - annualized (approx * sqrt(252) for scale)
    rmse = np.sqrt(mean_squared_error(true_s, model_s)) * np.sqrt(252)

    # 4. Annualized Tracking Error (Std Dev of residuals)
    te = residuals.std() * np.sqrt(252)

    return {
        "Model": model_name,
        "Correlation": round(corr, 4),
        "R-Squared": round(r2, 4),
        "RMSE (Ann.)": round(rmse, 4),
        "Tracking Error (Ann.)": round(te, 4)
    }

# ------------------------------------------------------------
# 2. Prepare Data
# ------------------------------------------------------------
# A. STATIC MODEL (MA1)
# Re-calculating static returns based on your 'weights' variable from earlier
# (Assuming 'weights' is the Series you created in section 6.0)
# If 'weights' isn't in memory, replace with your filtered top static weights
static_model_returns = (df_lreturns.values @ weights.values)
# Note: Ensure shapes align. If df_lreturns is (T, N) and weights is (N,), this works.
# Make it a Series for easier handling
static_model_series = pd.Series(static_model_returns, index=df_lreturns.index)
ma1_true_series = df_lreturns_all['MA1']

# B. DYNAMIC MODEL (MA2) - using your new 'reconstructed_returns'
# (Assuming you just ran the fixed code and have 'reconstructed_returns')
dynamic_model_series = reconstructed_returns
ma2_true_series = aligned_ma2

# ------------------------------------------------------------
# 3. Calculate & Display
# ------------------------------------------------------------
metrics_list = []

# Calculate Static
metrics_list.append(get_fit_metrics(ma1_true_series, static_model_series, "Static (MA1)"))

# Calculate Dynamic
metrics_list.append(get_fit_metrics(ma2_true_series, dynamic_model_series, "Dynamic (MA2)"))

```

```python
# Create DataFrame
df_metrics = pd.DataFrame(metrics_list)

# Display
print(df_metrics.to_string(index=False))

# Optional: Interpret the results
print("\n--- Interpretation ---")
print("High R   (>0.8) and Correlation (>0.9) indicate the reconstructed strategy is structurally identical.
      ")
print("Low Tracking Error (<0.05) implies the 'noise' we filtered out contributed very little to risk.")
```